

1 Computational Individuation: Isomorphism, 2 not Indeterminacy*

3 Colin Klein
The Australian National University
colin.klein@anu.edu.au

4 **Abstract**

5 A pair of arguments for the indeterminacy of physical logic gates
6 play an important role in debates about computational individuation.
7 These arguments purport to showing that one needs extrinsic, con-
8 textual factors to individuate even simple computations. One of the
9 arguments has a straightforward flaw. Reflection on that flaw shows
10 that there is a (mostly tacit) assumption on both sides of the debate:
11 that computations ought to be understood in a function-theoretic way.
12 I describe an alternative structure-theoretic understanding of the in-
13 trinsic mathematical structures instantiated by computations. I show
14 that on a structure-theoretic account, there is no indeterminacy. In-
15 stead, any logic gate belongs to a fully determinate equivalence class
16 under isomorphism. Some independent advantages of a structure-
17 theoretic account are also noted.

18
19 Keywords: computation; implementation; indeterminacy; mathemat-
20 ical structure; group theory

*Revision 2. Compiled January 5, 2025

1 The debate

Can we individuate what a physical system is computing—in the sense of Marr (1982)’s computational level of analysis—while appealing only to intrinsic facts about that system? Or must we appeal to extrinsic factors—*aetiology*, convention, intention—in order to do so? A pair of arguments have come to play an important role in this debate. They focus on the simple logic gates that are at the heart of modern computers. Each purports to show that there is a principled *indeterminacy* between two mathematical interpretations even of a single logic gate. The conclusion is that some kind of context is necessary to disambiguate physical computation.

If these arguments succeed, they are relevant to several current debates, including the viability of a mechanistic approach to computation (Piccinini, 2007, 2015; Coelho Mollo, 2018; Dewhurst, 2018a,b; Ritchie and Piccinini, 2019; Lee, 2021; Fresco and Milkowski, 2021; Anderson and Piccinini, 2024) and the degree to which one needs a ‘semantic’ approach to individuation (Shagrir, 2001; Piccinini, 2008; Harbecke and Shagrir, 2019; Shagrir, 2020; Papayannopoulos et al., 2022b). Indeterminacy arguments have played an especially important role in debates over whether intrinsic mathematical contents can form a basis for cognitive science. Notably, Egan (1995, 2010, 2014, 2017) has argued that there is a purely mathematical sense of computational individuation that corresponds to Marr’s computational level, and that is fixed by local, intrinsic factors. If intrinsic factors are insufficient to give a fully determinate mathematical story even about simple logic gates, Egan’s project would appear to be dead in the water (Facchin, 2022).

I am sympathetic to intrinsic mathematical individuation. This paper is a partial defence thereof. I begin by explicating the two indeterminacy arguments, noting that they are not equally strong. Having shown where they diverge, I clarify why that divergence matters. I then argue that many of the above debates rest on an unexamined premise—roughly, that computational individuation must be done solely in function-theoretic terms—and show that this is a premise that could, and probably should, be denied.

2 The arguments

Modern computers rely on logic gates: simple processors that are the physical implementation of Boolean functions. A basic prerequisite for computational individuation of a system of gates would be the ability to individuate what each gate is doing. Two arguments purport to show a principled indeterminacy in the interpretation of single gates.

58 *First argument* (Sprevak, 2010; Shagrir, 2020; Papayannopoulos et al.,
59 2022a): A set of logic gates corresponds to a Boolean expression. For any
60 Boolean expression containing the operators $\wedge, \vee, \neg$ and constants referring
61 to TRUE and FALSE, one may generate its *Boolean dual* by swapping each
62 instance of $\wedge$ and $\vee$ and swapping each constant for its compliment. The
63 result is also a perfectly good Boolean expression, but it is usually a *different*
64 Boolean expression. Yet neither the original nor its dual is obviously a more
65 principled interpretation of a gate. Hence it is indeterminate which function
66 gates compute, even when there is only one gate.

67 *Second argument* (Shagrir, 2001; Harbecke and Shagrir, 2019; Shagrir,
68 2020; Papayannopoulos et al., 2022b,a): Most physical logic gates are binary,
69 operating on two different voltage levels. Consider a hypothetical two-place
70 logic gate which is ternary-valued, i.e. operating on three voltage levels. I
71 will refer to these voltage levels as H, M , and L for high, medium, and low,
72 respectively.

73 The ternary gate outputs H when given (H, H) , L when given (L, L) , and
 M otherwise (see table 1). We may interpret the ternary gate as a Boolean

Gate operation	Interpretation 1	Interpretation 2
	$\{H, M\} = \text{TRUE}$ $\{L\} = \text{FALSE}$	$\{H\} = \text{TRUE}$ $\{M, L\} = \text{FALSE}$
$H, H \rightarrow H$	$T, T \rightarrow T$	$T, T \rightarrow T$
$M, \text{anything} \rightarrow M$	$T, (T \text{ or } F) \rightarrow T$	$F, (T \text{ or } F) \rightarrow F$
$L, L \rightarrow L$	$F, F \rightarrow F$	$F, F \rightarrow F$

Table 1: The operation of the ternary-valued gate and two interpretations. The middle row condenses 7 lines of the full truth table. The left column shows the operation of the gate on different inputs. The middle column shows the interpretation on which the gate performs $\vee$, while on the right it performs $\wedge$.

74 gate by grouping together voltage levels. How we group them matters. If we
75 treat $\{H, M\}$ as corresponding to TRUE and L as FALSE, then the ternary
76 gate corresponds to $\vee$. If we treat H as TRUE and $\{M, L\}$ as FALSE, then
77 the ternary gate corresponds to $\wedge$. So it is indeterminate which Boolean
78 function this gate computes, and this indeterminacy would be inherited by
79 any system of gates as well.
80

81 3 How they differ

82 The second argument has a problem that the first does not. Boolean algebra
83 requires a unary operator with the properties of truth-functional negation
84 (Halmos and Givant, 2009, 10). Without negation (either primitive or com-
85 posed), a set of gates cannot form a complete logical basis.¹

86 How could a ternary-valued gate that preserved Boolean negation work?
87 To be clear, the goal would be to produce a new ternary gate that is inter-
88 pretable as negation on both of the alternative mappings from $\{H, M, L\}$ to
89 $\{\text{TRUE}, \text{FALSE}\}$. For starters, note that it must take $H \rightarrow L$ and $L \rightarrow H$.
90 Aside from this being the obvious thing, note that if it did not—if, say, it
91 took $L \rightarrow M$ —then the interpretation that groups L and M as FALSE would
92 return FALSE given FALSE. Similarly so if $H \rightarrow M$. So on at least one
93 interpretation, the gate would fail to perform Boolean negation.

94 What should the gate output given M ? There are three possibilities (see
95 table 2). Consider first the case where the gate transforms $M \rightarrow H$. Then
96 on the interpretation that groups $\{H, M\}$, the gate will return TRUE given
97 an input of TRUE. So the gate will not perform negation on this interpre-
98 tation. In other words, while the original ternary gate may support a dual
99 interpretation for $\vee$ and $\wedge$, only one of those interpretations will support $\neg$
100 on this gate. Hence the system can't be indeterminate after all.

101 Similarly, if $M \rightarrow L$, then the interpretation that groups $\{M, L\}$ will
102 sometimes return FALSE given FALSE. If $M \rightarrow M$ then both interpretations
103 will sometimes fail. Indeed, note that not only does our putative ternary
104 negation gate fail to give the right answer on at least one grouping, but on
105 that interpretation it will fail to correspond to *any* Boolean function because
106 it will return different outputs for the same input.

107 Note that the point *isn't* that you can't construct something analogous
108 to negation in a ternary-valued system. Any of the three options in table 2
109 might be a useful way to do so, given the right context. The point is, rather,
110 that there is no option that ends up indeterminately mapped to Boolean
111 negation, because on at least one interpretation the gate would end up with
112 negation doing something that negation ought not to do.²

¹The 'either primitive or composed' qualifier is necessary because modern computers may use gates corresponding to a single Boolean function (like NAND) which form a full complete logical basis, or gates that are chosen for their useful properties (like XOR in a half adder). Note that if a single gate forms a complete basis, then it also allows the composition of negation. The argument following holds whether negation is primitive or composed; equivalently, the argument extends trivially to a proposed ternary NAND.

²I assume here that one wants a *single* gate that does negation on both interpretations, but what if we relax this restriction? An anonymous reviewer suggested the following ingenious construction. Consider *two* gates: Gate 1 takes $H \rightarrow L; M \rightarrow L; L \rightarrow M$, while

Transition	Interpretation 1	Interpretation 2
	$\{H, M\} = \text{TRUE}$ $\{L\} = \text{FALSE}$	$\{H\} = \text{TRUE}$ $\{M, L\} = \text{FALSE}$
$M \rightarrow H$	<u>$\neg T = T$</u>	$\neg F = T$
$M \rightarrow M$	<u>$\neg T = T$</u>	<u>$\neg F = F$</u>
$M \rightarrow L$	$\neg T = F$	<u>$\neg F = F$</u>

Table 2: Possibilities for the operation of a putative ternary-valued negation gate for M . The second two columns show the two possible interpretations, underlined expressions are ones where the gate does not perform negation on that interpretation.

By contrast, the first argument faces no such issue. The procedure for generating the Boolean dual did not mention negation, but it did not need to. Negation is *self-dual*; if we swap what stands for TRUE and FALSE (i.e. we take the complement of each), the truth table for $\neg$ is merely rearranged rather than altered (see Fresco et al. 2021, esp. pp12758ff) So any gate that performs negation on one interpretation will also perform negation on the dual.

4 Why it matters

One might object that the impossibility of indeterminate ternary negation does not touch the fact that the ternary gate so described is still indeter-

Gate 2 takes $H \rightarrow M; M \rightarrow H; L \rightarrow H$. On interpretation 1, Gate 1 performs negation and Gate 2 returns TRUE on all inputs. On Interpretation 2, Gate 1 returns FALSE on all inputs while Gate 2 performs negation.

While I cannot do full justice to the proposal here, I note two quick replies. First, having to appeal to two gates rather than one saps indeterminacy of much of its interest, especially given that the alternative interpretations are not logically interesting. This is a clear and important disanalogy with the first argument. Second, it is not clear to me that one can make a single system that uses both of the gates and does something interesting on both interpretations. The problem is that any time you do anything interesting with negation on one interpretation on one interpretation, you discard all interesting information on the other. So (for example) a NAND gate on interpretation 2 (which would form a complete basis) simply returns a tautology on interpretation 1. So does any *composition* of NAND gates: do anything interesting on interpretation 2, and interpretation 1 returns a tautology. Mixing gates should be worse, as it will throw away information for both interpretations. Investigating this further would, I think, require a full picture of the properties of the gates as they compose—and part of the point of the present paper is that the overall patterns of composition of gates are precisely what we ought to focus on, not just interpretations of single gates.

123 minate. One might respond that if you can't do negation, you're not even
124 playing the Boolean algebra game.

125 Something important lies behind this clash of intuitions: a common but
126 rarely articulated assumption about how mathematical functions themselves
127 ought to be individuated in the context of computational analysis. Most au-
128 thors assume that individuation at Marr's highest level should be understood
129 in *function-theoretic* terms. Egan is admirably clear about this, writing that
130 there is

131 ...an explanatory strategy that is pervasive in computational
132 cognitive science. I call the strategy *function-theoretic expla-*
133 *nation* and the mathematical characterisation that is central to
134 it *function-theoretic characterisation*... For present purposes we
135 can take functions to be mappings from sets (the arguments of
136 the function) to sets (its values). (Egan, 2017, 146).

137 What it is to assign a computation, says Egan, is to demonstrate a mapping
138 between elements. That is what underlies the intuition that the ternary
139 gate could be interpreted in a Boolean way, even though one could not do
140 consistently indeterminate ternary negation. After all, the ternary gate in
141 table 1 still performs the right kind of mapping for $\wedge$ and $\vee$. And even
142 the responses to indeterminacy that try to preserve intrinsic individuation—
143 whether at the physical (Dewhurst, 2018b) or functional (Coelho Mollo, 2018)
144 level—still focus on mappings from a set of states to a set of states.³

145 Yet a function-theoretic characterisation of the computational level is far
146 from obvious. For one, there are more functions than there are computable
147 functions (Sipser, 2013). For another, there are physical transformations
148 that can be mapped to functions (like consulting lookup tables) that don't
149 count as computing that function. For another, focusing purely on functions
150 cannot individuate architectures with very different power (Klein, 2019). At
151 the very least, then, computational analysis seems to require functions plus
152 some further mathematical structure.

153 A natural way to think about this further mathematical structure is in
154 terms of a set of primitive *operations* on a set of elements plus rules for how
155 those operations compose. When we talk about what a system is doing, we
156 need to preserve relationships between operators, not just between elements.

³That said, note that Anderson and Piccinini (2024) reaches a similar conclusion despite being mapping-friendly; see especially p129–130. The convergence suggests that a suitably sophisticated mapping account might also be able to avoid some kinds of indeterminacy; the degree to which such an account is compatible with a structure-theoretic one remains a topic for further research. Thanks to an anonymous reviewer for bringing my attention to this work.

157 Call this a *structure-theoretic* characterisation of the computational: we char-
 158 acterise a computation in terms of the primitive operations, the elements they
 159 operate on, and the composition relationships that hold between them.⁴ This
 160 is the notion of computation that is embedded in the linkage between com-
 161 putation and recursively composed functions (Boolos et al., 2007). This is
 162 also one reason why it is important that a system of logic gates be able to
 163 form a complete basis (at least in principle) if it is to count as part of a com-
 164 puting system: logical operations are interesting mainly insofar as they can
 165 be composed together in appropriately complex ways to form computations.
 166 If you can't do negation you can't make anything very interesting.

167 One of the best statements of a structure-theoretic account is actually
 168 found in Marr's initial discussion of computational analysis. We may un-
 169 derstand a cash register at many levels, Marr says, but the "most abstract"
 170 level asks "*what* the device does and *why*" (Marr, 1982, 22). What it does is
 171 arithmetic, he says, and to understand that we must consider first consider
 172 what addition is:

173 Addition is a mapping, usually denoted by $+$, from pairs of num-
 174 bers into single numbers; for example, $+$ maps the pair $(3, 4)$ to
 175 7, and I shall write this in the form $(3 + 4) = 7$. Addition has a
 176 number of abstract properties, however. It is commutative: both
 177 $(3 + 4)$ and $(4 + 3)$ are equal to 7; and associative: the sum of
 178 $3 + (4 + 5)$ is the same as the sum of $(3 + 4) + 5$. Then there is
 179 the unique distinguished element, zero, the adding of which has
 180 no effect: $(4 + 0) \rightarrow 4$. Also, for every number there is a unique
 181 "inverse," written (-4) in the case of 4, which when added to the
 182 number gives zero: $[4 + (-4)] \rightarrow 0$. (Marr, 1982, 22)

183 The same expository structure is duplicated immediately afterwards when
 184 he discusses why the cash register counts as performing addition: if you buy
 185 nothing it costs nothing, buying nothing plus something costs the same as
 186 something, and so on, each statement about prices mirroring a statement
 187 about addition. He concludes that since this mathematical structure is what
 188 defines the operation of addition, and the operations of the cash register
 189 faithfully mirrors that structure (Marr, 1982, 23), that settles the question
 190 of which computation is at work. The combination of these two claims—

⁴A small but important clarification: function composition is an atemporal mathemati-
 cal operation that places (at best) only logical constraints on the order in which operations
 are actually done. So talk of composition still occurs at Marr's computational level, rather
 than the algorithmic level. This can be obscured when talking about logic gates, because
 they are so basic that there's very little room between the formal operation of composition
 and the causal relationships that chain together primitives into algorithms.

191 one about the structure of addition and one about the way this structure is
192 mirrored in the basic operations of the cash register—together constitute the
193 computational theory of the domain.

194 Note that Marr’s characterisation is not a function-theoretic one: he’s not
195 content just to say that addition is a function that takes x and y to $x + y$.
196 The extra clauses in Marr’s definition make it clear that he’s evoking the
197 textbook definition of a mathematical group, which is a structure-theoretic
198 notion *par excellence*. A group is a set S of elements closed under a binary
199 operation $*$, where $*$ is associative; there is an identity element $e \in S$ s.t.
200 for any element $r \in S$, $e * r = r * e = r$; and for each element r there is an
201 element r' s.t. $r * r' = e$. Note that Marr has carefully demonstrated that
202 each of the group axioms hold.

203 Groups can be further distinguished by other properties they have—the
204 cardinality of the set, additional features of the operator (like commutativity
205 in the case of addition), the order of different elements, and so on. This
206 structure is in turn sufficient to distinguish the group: if you know enough
207 about how the operators compose, then you know enough to know which
208 group it is.

209 Now, not all computations will involve mappings to groups (Boolean alge-
210 bras have a different structure, for example). But Marr’s insight is that any
211 interesting set of operations will have some interesting mathematical struc-
212 ture on the operators, and it’s that structure that we look to in order to fix
213 what the computation is doing.⁵

214 And that is in turn why a structure-theoretic notion of computation balks
215 at the second argument for indeterminacy: whatever the ternary gate is
216 doing, we know we could not possibly find a consistent negation operator on
217 both interpretations. That’s enough to kill the interpretation that would go
218 through on a merely function-theoretic account. A function-theoretic account
219 can interpret operations piecemeal. Structure-theoretic accounts do not.

⁵There is of course a further question about what that structure is in any particular computational system. The broadest thing we probably can say—that there is a tight relationship between finite automata and semigroups (Kalman et al., 1969, ch6-9)—is relatively weak, so the particulars will matter if we want to say more. Expanding to the argument to analog (Maley, 2011) computations would also require considerations of continuous groups. One promising topic for future research involves the extent to which simple computational structures compose into more complex ones, which is a rich and well-developed part of group theory.

220 5 Upshots

221 Suppose one were to adopt a structure-theoretic account of individuation.
222 Aside from knocking down an counterargument, would there be any conse-
223 quences for the debate over intrinsic computational individuation? After all,
224 the first argument still appears to work, and one counterexample is enough.
225 I think it does make a difference. I conclude with some reflections about
226 what a structure-theoretic account might say to the more general question
227 of indeterminacy.

228 The structure-theoretic conception of computation is, in one sense, a more
229 demanding account of individuation: it requires a system to support a whole
230 set of operations. However, structure-theoretic notions can also be more
231 permissive in another sense.

232 Consider the following three mathematical groups: $\{0, 1, 2, 3\}$ equipped
233 with addition *mod* 4 (the cyclic group of integers $\mathbb{Z}_4$); $\{1, 3, 7, 9\}$ with mul-
234 tiplication *mod* 10; and $\{1, -1, i, -i\}$ with ordinary multiplication. The
235 groups have different underlying sets, and so their operators are function-
236 theoretically distinct. Yet each group is *isomorphic* to each of the others.
237 Anything you want to prove about one that holds on structural grounds will
238 hold of the other, because they have the same structure.

239 From a group-theoretic perspective, we can thus say that these groups are
240 *equivalent* where the equivalence classes are determined by isomorphism.
241 Once we have isomorphism in hand, the particulars of the operations can fall
242 away. One way to see that the three groups above are equivalent is that they
243 all have four elements and each has an element of order 4. That, combined
244 with the fact that they obey the group axioms, is sufficient to establish their
245 isomorphism. It is also sufficient to distinguish each from the other group
246 with four elements, V_4 , which has no element with an order greater than 2.
247 In other words, there are structural facts about the relationship between the
248 operators that allow us to abstract away from the details that a function-
249 theoretic account would center.

250 Isomorphism is what gives interpretive indeterminacy its philosophical
251 punch. For if A and B are isomorphic, there is a sense in which they are
252 ‘mere relabelings’ of one another. A Boolean expression and its dual capture
253 this feeling nicely: if we’d decided to swap the symbols for TRUE and FALSE
254 and the labels on the gates, we’d have another perfectly good interpretation.
255 The isomorphism between the two is precisely what makes it seem arbitrary
256 which one we pick.

257 Yet at this point, a structure-theoretic internalist might well draw the
258 opposite lesson. Why should it matter what we’ve decided to call our ele-
259 ments? The lesson is not that a set of gates is hopelessly indeterminate.

260 Rather, there is a fully *determinate* equivalence class to which we can assign
 261 both the canonical and dual interpretations of a set of gates, and it is that
 262 equivalence class that matters for computational individuation. Of course,
 263 for some purposes we might want to say more. If we are giving what Egan
 264 (1995) calls a cognitive gloss on the intrinsic mathematical state of affairs, for
 265 example, we might want a more fine-grained distinction by function alone.
 266 A structure-theoretic account nevertheless gives us a perfectly determinate
 267 way to individuate computational structures.

268 One might object that the move away from a function-theoretic charac-
 269 terisation feels alien: surely, when we talk about a computation, a basic
 270 thing we do is talk about the mathematical function we want to compute!
 271 Yet standard practice is not an unequivocal guide. One way to demonstrate
 272 an equivalence class we care about is to produce a member of it. Picking
 273 a member is a way both to locate an equivalence class and to specify an
 274 intended interpretation. Yet for many purposes the intended interpretation
 275 can be ignored. This is common in abstract algebra, where one often talks
 276 about an equivalence class by picking a single member (like $\mathbb{Z}_4$) about which
 277 it is easy to reason: talk of $\mathbb{Z}_4$ then stands in for talk of any of the groups
 278 isomorphic to it.

279 Given any group, of course, there are indefinitely many other groups iso-
 280 morphic to it. I am not concerned by that, any more than I am by the
 281 idea that there are indefinitely many physical ways to realise a computation.
 282 But if that attitude strikes you as too permissive, one way to scale it back
 283 would be to limit the equivalence class to what are sometimes called *canoni-*
 284 *cal* isomorphisms. Roughly speaking, a canonical isomorphism is one where
 285 there are no choices to make about what maps to what. Boolean duals are
 286 canonically isomorphic, as are many other mathematical duals.⁶

287 In sum, the function-theoretic approach to computational analysis focuses
 288 only on relationships between elements. Structure-theoretic accounts, like
 289 that proposed by Marr, treat computational individuation as a matter of the
 290 right relationships between basic building blocks of elements, operations, and
 291 composition. A structure-theoretic account sheds light on why the two inde-
 292 terminacy arguments diverge. It is not merely mappings between elements
 293 that matter for individuation, but also the operators and their properties
 294 when composed. When we focus on that structure, equivalence up to iso-
 295 morphism may be entirely sufficient for individuation—and so sufficient for
 296 a purely internalist story about computational individuation.

⁶For example, a finite vector space's double dual vector space is canonically isomorphic, unlike any of its dual vector spaces, and so does not require a choice of basis (Axler, 2015, 116).

References

- Anderson, N. G. and Piccinini, G. (2024). *The physical signature of computation: A robust mapping account*. Oxford University Press.
- Axler, S. J. (2015). *Linear Algebra Done Right*. Springer, New York, 3rd edition.
- Boolos, G., Burgess, J. P., and Jeffrey, R. C. (2007). *Computability and Logic*. Cambridge University Press, New York, 5th edition.
- Coelho Mollo, D. (2018). Functional individuation, mechanistic implementation: The proper way of seeing the mechanistic view of concrete computation. *Synthese*, 195(8):3477–3497.
- Dewhurst, J. (2018a). Computing mechanisms without proper functions. *Minds and Machines*, 28(3):569–588.
- Dewhurst, J. (2018b). Individuation without representation. *The British Journal for the Philosophy of Science*, 69:103–116.
- Egan, F. (1995). Computation and content. *The Philosophical Review*, 104(2):181–203.
- Egan, F. (2010). Computational models: A modest role for content. *Studies in History and Philosophy of Science Part A*, 41(3):253–259.
- Egan, F. (2014). How to think about mental content. *Philosophical Studies*, 170:115–135.
- Egan, F. (2017). Function-theoretic explanation and the search for neural mechanisms. In Kaplan, D. M., editor, *Explanation and Integration in Mind and Brain Science*, pages 145–163. Oxford University Press, Oxford.
- Facchin, M. (2022). Troubles with mathematical contents. *Philosophical Psychology*, <https://doi.org/10.1080/09515089.2022.2119952>.
- Fresco, N., Copeland, B. J., and Wolf, M. J. (2021). The indeterminacy of computation. *Synthese*, 199:12753–12775.
- Fresco, N. and Milkowski, M. (2021). Mechanistic computational individuation without biting the bullet. *The British Journal for the Philosophy of Science*, 72(2):431–438.
- Halmos, P. and Givant, S. (2009). *Introduction to Boolean algebras*. Springer, New York.

- 329 Harbecke, J. and Shagrir, O. (2019). The role of the environment in compu-
330 tational explanations. *European Journal for Philosophy of Science*, 9:1–19.
- 331 Kalman, R. E., Falb, P. L., and Arbib, M. A. (1969). *Topics in mathematical*
332 *system theory*. McGraw-Hill, New York.
- 333 Klein, C. (2019). Computation, consciousness, and “Computation and con-
334 sciousness”. In Sprevak, M. and Colombo, M., editors, *The Routledge*
335 *Handbook Of The Computational Mind*, pages 297–309. Routledge.
- 336 Lee, J. (2021). Mechanisms, wide functions, and content: Towards a compu-
337 tational pluralism. *The British Journal for the Philosophy of Science*.
- 338 Maley, C. J. (2011). Analog and digital, continuous and discrete. *Philosophical*
339 *Studies*, 155:117–131.
- 340 Marr, D. (1982). *Vision: A computational investigation into the human*
341 *representation and processing of visual information*. WH Freeman, New
342 York.
- 343 Papayannopoulos, P., Fresco, N., and Shagrir, O. (2022a). Computational
344 indeterminacy and explanations in cognitive science. *Biology & Philosophy*,
345 37(6):47.
- 346 Papayannopoulos, P., Fresco, N., and Shagrir, O. (2022b). On two different
347 kinds of computational indeterminacy. *The Monist*, 105(2):229–246.
- 348 Piccinini, G. (2007). Computing mechanisms. *Philosophy of Science*,
349 74(4):501–526.
- 350 Piccinini, G. (2008). Computation without representation. *Philosophical*
351 *Studies*, 137(2):205–241.
- 352 Piccinini, G. (2015). *Physical computation: A mechanistic account*. Oxford
353 University Press, Oxford.
- 354 Ritchie, J. B. and Piccinini, G. (2019). Computational implementation. In
355 Sprevak, M. and Colombo, M., editors, *The Routledge Handbook Of The*
356 *Computational Mind*, pages 192–204. Routledge.
- 357 Shagrir, O. (2001). Content, computation and externalism. *Mind*,
358 110(438):369–400.
- 359 Shagrir, O. (2020). In defense of the semantic view of computation. *Synthese*,
360 197:4083–4108.

- 361 Sipser, M. (2013). *Introduction to the Theory of Computation*. Cengage
362 Learning, Boston, 3 edition.
- 363 Sprevak, M. (2010). Computation, individuation, and the received view
364 on representation. *Studies in History and Philosophy of Science Part A*,
365 41(3):260–270.